Dynamic Programming - LeetCode Problems

Problem statement | Examples | DP idea | Java code | Complexity

~Ninganna

Your Solved DP Pattern Set

Pattern	LeetCode
LIS	300
LCS	1143
LCS	583
Common Substring	718
LPS	516
LPS	1312
Grid / Minimum Cost Path	64
Grid / Minimum Cost Path	62
Grid / Minimum Cost Path	63
Edit Distance	72
Subset Sum	416
Maximal Square	221
Maximal Square	1277

LIS

1. Longest Increasing Subsequence - LeetCode 300

Problem statement: Given nums, find the length of the longest strictly increasing subsequence.

Example:
Input: [10,9,2,5,3,7,101,18] -> Output: 4. One LIS is [2,3,7,101].

DP idea: dp[i] = LIS length ending at nums[i]. If nums[j] < nums[i], nums[i] can extend dp[j].

Java implementation:

```java
class Solution {
    public int lengthOfLIS(int[] nums) {
        int n = nums.length;
        int[] dp = new int[n];

        for (int i = 0; i < n; i++) {
            dp[i] = 1;
        }

        int max = 1;

        for (int i = 1; i < n; i++) {
            for (int j = 0; j < i; j++) {
                if (nums[j] < nums[i]) {
                    dp[i] = Math.max(dp[i], dp[j] + 1);
                }
            }
            max = Math.max(max, dp[i]);
        }

        return max;
    }
}
```

Complexity Time: O(n^2) Space: O(n)

LCS

2. Longest Common Subsequence - LeetCode 1143

Problem statement: Given text1 and text2, return the length of their longest common subsequence.

Example:
Input: text1 = "abcde", text2 = "ace" -> Output: 3. LCS = "ace".

DP idea: dp[i][j] = LCS of the first i and first j characters. Match -> diagonal + 1. Mismatch -> max(top, left).

Java implementation:

```java
class Solution {
    public int longestCommonSubsequence(String text1, String text2) {
        int m = text1.length();
        int n = text2.length();

        int[][] dp = new int[m + 1][n + 1];

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (text1.charAt(i - 1) == text2.charAt(j - 1)) {
                    dp[i][j] = dp[i - 1][j - 1] + 1;
                } else {
                    dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
                }
            }
        }

        return dp[m][n];
    }
}
```

Complexity Time: O(m x n) Space: O(m x n)

3.Delete Operation for Two Strings - LeetCode 583

Problem statement: Return the minimum deletions from either string needed to make word1 and word2 equal.

Example:
Input: word1 = "sea", word2 = "eat" -> Output: 2.

DP idea: Find LCS length L. Delete m-L characters from word1 and n-L from word2, so answer = m+n-2L.

Java implementation:

```java
class Solution {
    public int minDistance(String word1, String word2) {
        int m = word1.length();
        int n = word2.length();

        int[][] dp = new int[m + 1][n + 1];

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (word1.charAt(i - 1) == word2.charAt(j - 1)) {
                    dp[i][j] = dp[i - 1][j - 1] + 1;
                } else {
                    dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
                }
            }
        }

        int lcs = dp[m][n];
        return (m - lcs) + (n - lcs);
    }
}
```

Complexity Time: O(m x n) Space: O(m x n)

Common Substring

4. Maximum Length of Repeated Subarray - LeetCode 718

Problem statement: Find the maximum length of a contiguous subarray appearing in both arrays.

Example:
Input: [1,2,3,2,1], [3,2,1,4,7] -> Output: 3. Common subarray = [3,2,1].

DP idea: dp[i][j] = common contiguous length ending at the two current elements. Match -> diagonal + 1; mismatch -> 0.

Java implementation:

```java
class Solution {
    public int findLength(int[] nums1, int[] nums2) {
        int m = nums1.length;
        int n = nums2.length;

        int[][] dp = new int[m + 1][n + 1];
        int max = 0;

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (nums1[i - 1] == nums2[j - 1]) {
                    dp[i][j] = dp[i - 1][j - 1] + 1;
                    max = Math.max(max, dp[i][j]);
                } else {
                    dp[i][j] = 0;
                }
            }
        }

        return max;
    }
}
```

Complexity Time: O(m x n) Space: O(m x n)

LPS

5. Longest Palindromic Subsequence - LeetCode 516

Problem statement: Return the length of the longest subsequence of s that is also a palindrome.

Example:
Input: "bbbab" -> Output: 4. One LPS is "bbbb".

DP idea: dp[i][j] = LPS length inside s[i..j]. Single characters are 1. Fill i backwards because the transition uses i+1.

Java implementation:

```java
class Solution {
    public int longestPalindromeSubseq(String s) {
        int n = s.length();
        int[][] dp = new int[n][n];

        for (int i = 0; i < n; i++) {
            dp[i][i] = 1;
        }

        for (int i = n - 2; i >= 0; i--) {
            for (int j = i + 1; j < n; j++) {
                if (s.charAt(i) == s.charAt(j)) {
                    dp[i][j] = dp[i + 1][j - 1] + 2;
                } else {
                    dp[i][j] = Math.max(dp[i + 1][j], dp[i][j - 1]);
                }
            }
        }

        return dp[0][n - 1];
    }
}
```

Complexity Time: O(n^2) Space: O(n^2)

6.Minimum Insertion Steps to Make a String Palindrome - LeetCode 1312

Problem statement: Return the minimum insertions needed to make s a palindrome.

Example:
Input: "mbadm" -> Output: 2.

DP idea: Use LPS. If LPS has length L, minimum insertions = n-L.

Java implementation:

```java
class Solution {
    public int minInsertions(String s) {
        int n = s.length();
        int[][] dp = new int[n][n];

        for (int i = 0; i < n; i++) {
            dp[i][i] = 1;
        }

        for (int i = n - 2; i >= 0; i--) {
            for (int j = i + 1; j < n; j++) {
                if (s.charAt(i) == s.charAt(j)) {
                    dp[i][j] = dp[i + 1][j - 1] + 2;
                } else {
                    dp[i][j] = Math.max(dp[i + 1][j], dp[i][j - 1]);
                }
            }
        }

        int lps = dp[0][n - 1];
        return n - lps;
    }
}
```

Complexity Time: O(n^2) Space: O(n^2)

Grid / Minimum Cost Path

7. Minimum Path Sum - LeetCode 64

Problem statement: Find the minimum sum path from top-left to bottom-right, moving only right or down.

Example:
Input: [[1,3,1],[1,5,1],[4,2,1]] -> Output: 7.

DP idea: dp[i][j] = minimum cost to reach (i,j). Add the current value to min(top,left).

Java implementation:

```java
class Solution {
    public int minPathSum(int[][] grid) {
        int m = grid.length;
        int n = grid[0].length;

        int[][] dp = new int[m][n];

        dp[0][0] = grid[0][0];

        for (int i = 1; i < m; i++) {
            dp[i][0] = dp[i - 1][0] + grid[i][0];
        }

        for (int j = 1; j < n; j++) {
            dp[0][j] = dp[0][j - 1] + grid[0][j];
        }

        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                dp[i][j] = grid[i][j] +
                           Math.min(dp[i - 1][j], dp[i][j - 1]);
            }
        }

        return dp[m - 1][n - 1];
    }
}
```

Complexity Time: O(m x n) Space: O(m x n)

8.Unique Paths - LeetCode 62

Problem statement: Count paths from top-left to bottom-right in an m x n grid, moving only right or down.

Example:
Input: m = 3, n = 7 -> Output: 28.

DP idea: dp[i][j] = paths reaching the cell. Each inner cell gets paths from top + left.

Java implementation:

```java
class Solution {
    public int uniquePaths(int m, int n) {
        int[][] dp = new int[m][n];

        for (int j = 0; j < n; j++) {
            dp[0][j] = 1;
        }

        for (int i = 0; i < m; i++) {
            dp[i][0] = 1;
        }

        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
            }
        }

        return dp[m - 1][n - 1];
    }
}
```

Complexity Time: O(m x n) Space: O(m x n)

9.Unique Paths II - LeetCode 63

Problem statement: Count paths in a grid with obstacles; move only right or down. A 1-cell is blocked.

Example:
Input: [[0,0,0],[0,1,0],[0,0,0]] -> Output: 2.

DP idea: Same as Unique Paths, but an obstacle has 0 paths. In the first row/column, an obstacle blocks all cells after it.

Java implementation:

```java
class Solution {
    public int uniquePathsWithObstacles(int[][] obstacleGrid) {
        int m = obstacleGrid.length;
        int n = obstacleGrid[0].length;

        int[][] dp = new int[m][n];

        if (obstacleGrid[0][0] == 1) {
            return 0;
        }

        dp[0][0] = 1;

        for (int i = 1; i < m; i++) {
            if (obstacleGrid[i][0] == 0) {
                dp[i][0] = dp[i - 1][0];
            }
        }

        for (int j = 1; j < n; j++) {
            if (obstacleGrid[0][j] == 0) {
                dp[0][j] = dp[0][j - 1];
            }
        }

        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                if (obstacleGrid[i][j] == 1) {
                    dp[i][j] = 0;
                } else {
                    dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
                }
            }
        }

        return dp[m - 1][n - 1];
    }
}
```

Complexity Time: O(m x n) Space: O(m x n)

Edit Distance

10. Edit Distance - LeetCode 72

Problem statement: Convert word1 into word2 using minimum insert, delete, or replace operations.

Example:
Input: word1 = "horse", word2 = "ros" -> Output: 3.

DP idea: dp[i][j] = minimum operations for the first i and j characters. Match -> diagonal. Otherwise choose replace, delete, or insert + 1.

Java implementation:

```java
class Solution {
    public int minDistance(String word1, String word2) {
        int m = word1.length();
        int n = word2.length();

        int[][] dp = new int[m + 1][n + 1];

        for (int i = 0; i <= m; i++) {
            dp[i][0] = i;
        }

        for (int j = 0; j <= n; j++) {
            dp[0][j] = j;
        }

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (word1.charAt(i - 1) == word2.charAt(j - 1)) {
                    dp[i][j] = dp[i - 1][j - 1];
                } else {
                    dp[i][j] = Math.min(
                        dp[i - 1][j - 1],
                        Math.min(dp[i - 1][j], dp[i][j - 1])
                    ) + 1;
                }
            }
        }

        return dp[m][n];
    }
}
```

Complexity Time: O(m x n) Space: O(m x n)

Subset Sum

11. Partition Equal Subset Sum - LeetCode 416

Problem statement: Determine whether nums can be split into two subsets having equal sum.

Example:
Input: [1,5,11,5] -> Output: true. Split as [1,5,5] and [11].

DP idea: If total sum is odd, return false. Otherwise target = total/2 and dp[i][j] says whether the first i numbers can make sum j.

Java implementation:

```java
class Solution {
    public boolean canPartition(int[] nums) {
        int n = nums.length;
        int totalSum = 0;

        for (int i = 0; i < n; i++) {
            totalSum += nums[i];
        }

        if (totalSum % 2 != 0) {
            return false;
        }

        int target = totalSum / 2;
        boolean[][] dp = new boolean[n + 1][target + 1];

        for (int i = 0; i <= n; i++) {
            dp[i][0] = true;
        }

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= target; j++) {
                if (nums[i - 1] <= j) {
                    dp[i][j] = dp[i - 1][j] ||
                               dp[i - 1][j - nums[i - 1]];
                } else {
                    dp[i][j] = dp[i - 1][j];
                }
            }
        }

        return dp[n][target];
    }
}
```

Complexity Time: O(n x target) Space: O(n x target)

Maximal Square

12. Maximal Square - LeetCode 221

Problem statement: Find the largest square containing only 1s in a binary matrix and return its area.

Example:
Input: [[1,0,1],[1,1,1],[1,1,1]] -> Output: 4. Largest side = 2.

DP idea: dp[i][j] = largest all-1 square side ending at matrix[i-1][j-1]. For 1: min(top,left,diagonal)+1.

Java implementation:

```java
class Solution {
    public int maximalSquare(char[][] matrix) {
        int m = matrix.length;
        int n = matrix[0].length;

        int[][] dp = new int[m + 1][n + 1];

        int max = 0;

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (matrix[i - 1][j - 1] == '1') {
                    dp[i][j] = Math.min(
                        dp[i - 1][j],
                        Math.min(
                            dp[i][j - 1],
                            dp[i - 1][j - 1]
                        )
                    ) + 1;

                    max = Math.max(max, dp[i][j]);
                } else {
                    dp[i][j] = 0;
                }
            }
        }

        return max * max;
    }
}
```

Complexity Time: O(m x n) Space: O(m x n)

13.Count Square Submatrices with All Ones - LeetCode 1277

Problem statement: Count all square submatrices containing only 1s. Squares of every size are counted.

Example:
Input: [[1,1,0],[1,1,1],[0,1,1]] -> Output: 9.

DP idea: Use the same square DP as 221. A dp value of k means k squares end at that cell, so sum all dp values.

Java implementation:

```java
class Solution {
    public int countSquares(int[][] matrix) {
        int m = matrix.length;
        int n = matrix[0].length;

        int[][] dp = new int[m + 1][n + 1];

        int count = 0;

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (matrix[i - 1][j - 1] == 1) {
                    dp[i][j] = Math.min(
                        dp[i - 1][j],
                        Math.min(
                            dp[i][j - 1],
                            dp[i - 1][j - 1]
                        )
                    ) + 1;
                } else {
                    dp[i][j] = 0;
                }

                count += dp[i][j];
            }
        }

        return count;
    }
}
```

Complexity Time: O(m x n) Space: O(m x n)